



COMP 4300

Intro to Game Programming

Lecture 9
Game Actions
Replay System

Game Inputs & Actions

- Games are **interactive**, and players get to influence the gameplay via **input devices**
- **Inputs** are given by the player of a game to influence what entities in the game do
 - Example: Press Key, Mouse Click, Joystick
- The **result** of an input is typically an **action** that the player wants to happen in the game
 - Example: Jump, Walk Right, Shoot
- Actions can also be called **Commands**

Game Inputs & Actions

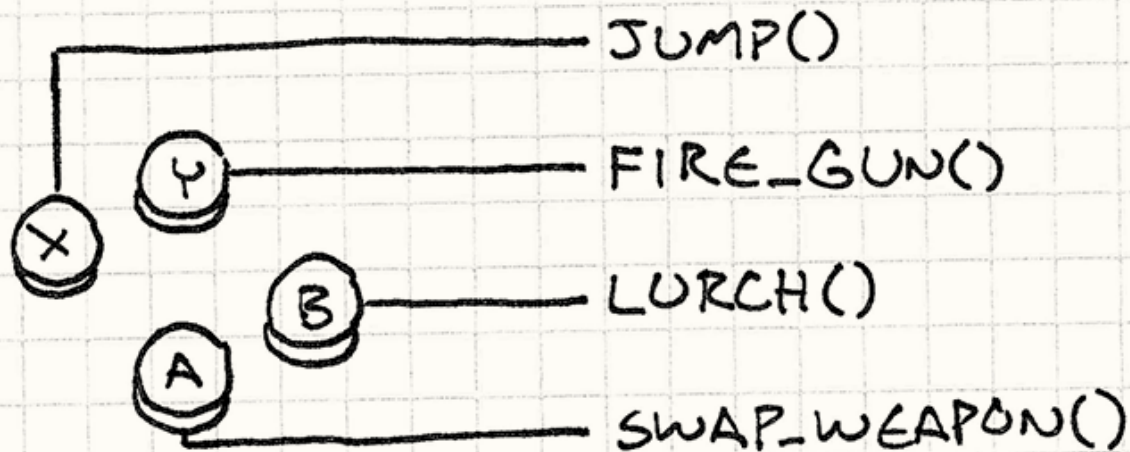
- It doesn't really matter to the player which Input **causes** an Action, as long as it's **easy to perform**
- Similarly, the game engine shouldn't care which **input device** caused an action to be triggered
- The game engine should only care about the **action logic** itself and how to carry it out
- So ideally, we would like to **decouple** the action logic from the inputs that triggered them

Game Action Implementation

- So far, we have been performing actions **directly** inside the sfml event loop
- When a specific key is pressed, specific game logic gets called
- In our current engine, we can't separate the two, or re-map keys, etc
- Let's change that

Assignment 2 Event Code

```
// this event is triggered when a key is pressed
if (event.type == sf::Event::KeyPressed)
{
    switch (event.key.code)
    {
        case sf::Keyboard::Escape: m_running = false;           break;
        case sf::Keyboard::W: player()->get<CInput>().up = true;   break;
        case sf::Keyboard::A: player()->get<CInput>().left = true;  break;
        case sf::Keyboard::S: player()->get<CInput>().down = true;  break;
        case sf::Keyboard::D: player()->get<CInput>().right = true; break;
        case sf::Keyboard::P: setPaused(!m_paused);               break;
        default: break;
    }
}
```



Game Action Implementation

- In Assignment 2, all of our input handling and Systems logic was done in the **Game** class, since we only had one Scene type
- Moving forward, we will have a **GameEngine** class which can manage any number of Scene objects
- Let's recall

GameEngine Class

- Stores **top level** game data
 - Assets, sf::Window, Scenes
- Performs top level functionality
 - Changing Scenes, Handling Input
- Run() contains game **main loop**
- GameEngine pointer will be passed to Scenes in constructor
 - Access to: assets, window, scenes

GameEngine	
- m_scenes:	map<str,Scene>
- m_window:	sf::Window
- m_assets:	Assets
- m_scene:	string
- m_running:	bool
- init():	void
+ update():	void
+ run():	void
+ update():	void
+ quit():	void
+ changeScene<T>():	void
+ getAssets():	Assets&
+ window():	sf::Window&
+ sUserInput():	void

Scene Base Class

- Stores **common Scene data**
 - Entities, Frame Count, Actions
- Scene-specific functionality is carried out in Derived classes
- Base Scene class is **Abstract**, cannot be instantiated
- `simulate()` calls derived scene's `update()` a given number of times (will be useful later)

Scene (Abstract)

Scene (Abstract)	
- m_game:	GameEngine*
- m_entities:	EntityMgr
- m_frame:	int
- m_actionMap:	map<int,str>
- m_paused:	bool
+ update():	void=0
+ sDoAction(action):	void=0
+ sRender():	void=0
+ simulate(int):	void
+ doAction(action):	void
+ registerAction(a):	void

Scene Derived Class

- Stores **Scene-specific** data
 - Level, Player Pointer, Config
- Scene-specific Systems are defined within the derived class
- Some Scene derived classes may require quite different systems based on functionality
- **update(), sRender(), sDoAction()** must be implemented for each Scene

Scene_Play : Scene

- m_level:	string
- m_config:	PlayerConfig
- init():	void
+ update():	void
- sAnimation():	void
- sMovement():	void
- sEnemySpawner():	void
- sCollision():	void
- sRender():	void
- sDoAction(action):	void
- sGUI():	void

New Action Implementation

- Let's try and implement a system where the Scene **only** knows the **type of action** the player wants to perform, and **doesn't care** about the input method
- In this way, actions could come from **anywhere**: keyboard, mouse, replay file, network stream, VR controller, etc

Action Class

- We will construct an **Action** class, which will store the type of action that the player wants the Scene to perform
- All user **input** will be handled within the GameEngine class, where Actions will be **created** and **sent** to the Scenes to have their logic carried out
- The Scene will then have a **doAction** function which performs logic based on a specific Action object

Action Class

- Actions will be given a **name** (jump, left, etc)
- Actions input via keyboard or controller will have two phases: **press** and **release**, which typically denote the start and end of action
- Actions will then have two main variables
 - Name: Name of the action (jump, shoot, etc)
 - Type: START or END

```
class Action
{
    std::string m_name  = "NONE";
    std::string m_type  = "NONE";

public:

    Action() = default;
    Action(const std::string& name, const std::string& type)
        : m_name(name), m_type(type) { }
    const std::string& name() const { return m_name; }
    const std::string& type() const { return m_type; }
};
```

Action Start / End

- Action Start = Press Button
- Action End = Release Button
- Example: {"Right", "Start"}
 - Player will run Right as if you held down the move right input button continuously
 - Will only stop moving on {"Right", "End"}

Mapping Keys to Actions

- We want to be able to **specify** which user input (ex: keyboard key) **maps** to a specific Action object
- In SFML, all keyboard keys have corresponding **Integer** values
- So let's use a `std::map<int, std::string>`
 - `int` = SFML **key code** for keyboard key
 - `string` = **name** of the action to be performed

Mapping Keys to Actions

- The Scene base class will store this map
 - `std::map<int, std::string> m_actionMap;`
- To **register** (map) a key to an Action, we create a function `registerAction(int, string)` which will insert the action name into the map
 - `m_actionMap[inputKey] = actionName;`
- Each Scene will now have **its own map** so we can have different actions available in each Scene

Action Registration

```
void Scene_Menu::init()
{
    registerAction(sf::Keyboard::W,      "UP");
    registerAction(sf::Keyboard::S,      "DOWN");
    registerAction(sf::Keyboard::D,      "PLAY");
    registerAction(sf::Keyboard::Escape, "QUIT");
}

void Scene::registerAction(int inputKey, const std::string& aName)
{
    m_actionMap[inputKey] = aName;
}
```

Creating Action Objects

- The **GameEngine** class will handle the actual user input key presses and **construct** the Action to send to the current Scene
- When a key is pressed, the GameEngine will **ask the Scene** if it has an action name **associated** with the key's integer value
- If the Scene has an associated Action, we create an Action object with the correct NAME and TYPE
 - KeyPress = START, KeyRelease = END

Creating Actions

```
if (event.type == sf::Event::KeyPressed || event.type == sf::Event::KeyReleased)
{
    // if the current scene does not have an action associated with this key, skip the event
    if (currentScene()->getActionMap().find(event.key.code) ==
        currentScene()->getActionMap().end())
    {
        continue;
    }

    // determine start or end action by whether it was key pres or release
    const std::string actionType = (event.type == sf::Event::KeyPressed) ? "START" : "END";

    // look up the action and send the action to the scene
    Action action(currentScene()->getActionMap().at(event.key.code), actionType);
    currentScene()->doAction(action);
}
```

Doing Scene Actions

- Now that we have constructed the Action object in the GameEngine, we **send it to the Scene** to have its logic performed
- This is done by the derived Scene class's `sDoAction(const Action & action)` function
- We read the name and the type of the action and perform its logic, **without knowing** the user input that created it

```
void Scene_Menu::sDoAction(const Action& action)
{
    if (action.type() == "START")
    {
        if (action.name() == "UP")
        {
            if (m_selectedMenuIndex > 0) { m_selectedMenuIndex--; }
            else { m_selectedMenuIndex = m_menuStrings.size() - 1; }
        }
        else if (action.name() == "DOWN")
        {
            m_selectedMenuIndex = (m_selectedMenuIndex + 1) % m_menuStrings.size();
        }
        else if (action.name() == "PLAY")
        {
            m_game.changeScene("PLAY", std::make_shared<Scene_Play>(
                m_game, m_levelPaths[m_selectedMenuIndex]));
        }
        else if (action.name() == "QUIT")
        {
            onEnd();
        }
    }
}
```

Alternate Architecture

- Another way to accomplish this would be to make each type of action a **derived class** of the base Action class
- Derived action classes would have a function that **carries out** the effects
- This would deviate from our ECS 'Systems' architecture, so not implemented here

Alternate Architecture

```
class ActionJump : Action
{

public:

    void execute(std::shared_ptr<Scene_Play> scene)
    {
        // Scene_Play would friend the Action class
        // to give access to its variables, entities, etc
        // Actions would then do the associated logic
    }

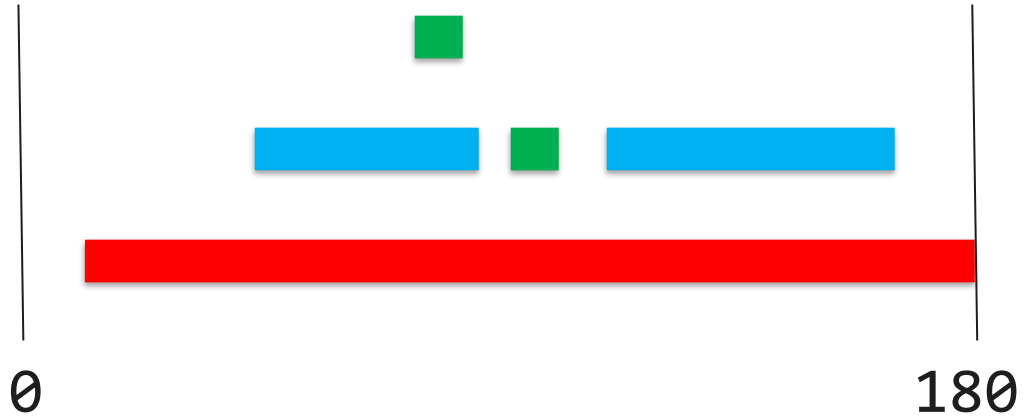
};
```

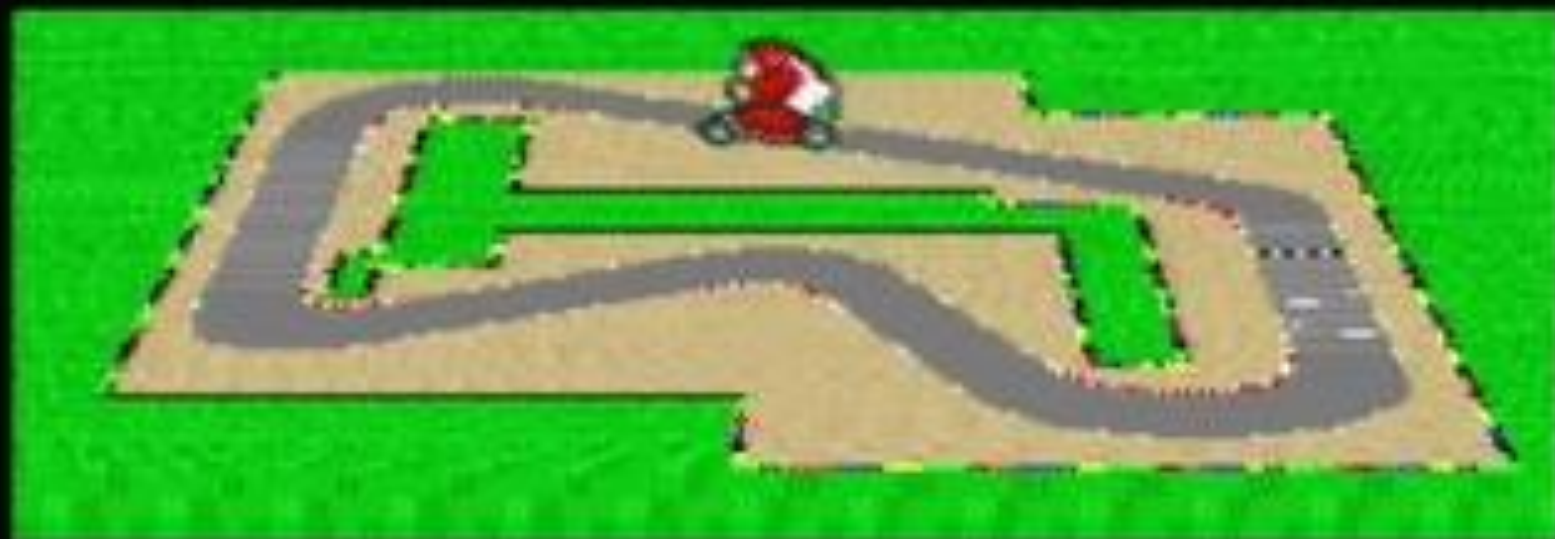
Replays

- Many games have functionality to be able to **record gameplay** in the form of **replays**
- Now that our Scenes don't care about **where an Action came from**, we can see how easy it would be to implement Replay files
- For example, just store our Action strings in a file along with the specific **game frame** that they were performed on. Then load this file in the GameEngine class and feed Actions to the Scene

Replay Example

30	RIGHT	START
60	JUMP	START
80	SHOOT	START
82	SHOOT	END
90	JUMP	END
100	SHOOT	START
105	SHOOT	END
120	JUMP	START
160	JUMP	END
180	RIGHT	END







COMP 4300 - Game Engine Architecture

GameEngine	Scene (Abstract Base)	Scene_Play : Scene	Scene_Menu : Scene	EntityManager	Entity
- scenes: map<string, Scene> - window: sf::RenderWindow - assets: Assets - currentScene: string - running: bool	- game: GameEngine * - entities: EntityManager - currentFrame: int - actionMap: map<int, string> - paused: bool - hasEnded: bool	- levelPath: string - player: Entity * - playerConfig: PlayerConfig	- menuStrings: vec<string> - menyText: sf::Text - levelPaths: vec<string> - menuIndex: int	- entities: vector<Entity *> - entityMap: map<string, Entity *> - toAdd: vector<Entity *>	- components: tuple<C1,C2..> - tag: string - active: bool - id: int
- init: void - currentScene(): Scene * + run(): void + update(): void + quit(): void + changeScene(scene): void + getAssets(): Assets & + window(): Window & + sUserInput(): void	+ update(): void = 0 + sDoAction(action): void = 0 + sRender(): void = 0 + simulate(int): void + doAction(action): void + registerAction(action): void	- init(): void + update(): void // Systems - sAnimation(): void - sMovement(): void - sEnemySpawner(): void - sCollision(): void - sRender(): void - sDoAction(action): void - sDebug(): void	- init(): void + update(): void // Systems - sRender(): void - sDoAction(action): void	- init: void + update(): void + addEntity(args): Entity * + getEntities(): vector<Entity *> & + getEntities(s): vector<Entity *> &	+ destroy: void + addComponent<C>: void + hasComponent<C>: bool + getComponent<C>: C & + removeComponent<C>: void

Assets	Animation	Vec2	Action	«interface» Component
- textures: map<string, sf::texture> - animations: map<string, Animation> - sounds: map<string, sf::Sound> - fonts: map<string, sf::Font>	- sprite: sf::Sprite - frameCount: int - currentFrame: int - speed: int - size: Vec2 - name: string	+ x: double + y: double	- name: string - type: string	
+ addTexture(name, path): void + addAnimation(name, Animation): void + addSound(name, path): void + addFont(name, path): void	+ update(): void + hasEnded(): void + getName(): string & + getSize(): Vec2 & + getSprite(): sf::Sprite &	+ operator ==: bool + operator !=: bool + operator +: Vec2 + operator -: Vec2 + operator *: Vec2 + operator /: Vec2	+ name(): string & + type(): string &	CTransform + pos: vec2 + velocity: vec2 + scale: vec2 + angle: double
+ getTexture(name): sf::Texture & + getAnimation(name): Animation & + getSound(name): sf::Sound & + getFont(name): sf::Font &		+ normalize(): void + length(): double	Physics +IsCollision(Entity, Entity): bool +IsIntersect(Line, Line): bool +IsInside(Vec2, Line): bool	CBoundingBox + size: vec2